

15 Days of Full-Stack Engineering

A day-by-day account of building one API and its Angular frontend from scratch – written to be explained out loud, in an interview, without notes.

ASP.NET Core 10 · Minimal APIs

EF Core + SQL Server

JWT · Entra ID · Authorization

OpenTelemetry · Serilog · Azure Monitor

Docker · Azure Container Apps

Angular 21 · Signals · Zoneless

PART ONE

Backend Foundations

Getting one small API right: dependency injection, domain modelling, authentication that actually resists a stolen token, two identity providers working side by side, and enough observability to diagnose a real production bug instead of guessing at it.

Day 1 – Setup & refactor

Day 2 – DI, domain, auth

Day 3 – Entra ID, authorization, CI

Day 4 – Observability

DAY 01 Two languages, one API, and a deliberately bad controller

The same tiny program in C# and TypeScript

Wrote the exact same "hello" program twice, once in C# and once in TypeScript run directly on Node, to compare what each runtime actually requires before you can run anything.

INTERVIEW ANGLE

C# needs a project file (`.csproj`) generated by `dotnet new` before it will run anything – it's a compiled, project-based ecosystem. Modern Node (24+) will run a `.ts` file directly with zero config: no `tsc` , no manifest. That's not "TypeScript is simpler than C#" – it's that Node grew a built-in TypeScript stripper, while .NET's whole tooling model still assumes a project.

QuotesApi – the project every later day builds on

Built a real REST API for storing quotes with ASP.NET Core's Minimal API style, EF Core, and SQLite: `GET /api/quotes` (paginated), `POST /api/quotes` , `GET /api/quotes/{id}` , `DELETE /api/quotes/{id}` . Every endpoint after this, for the next fourteen days, is built on top of this one project.

INTERVIEW ANGLE

Four things worth naming if asked "what did you put in from day one": EF Core migrations that apply automatically on startup (no manual schema step), a repository interface (`IQuoteRepository`) registered as scoped DI rather than calling `DbContext` directly from the endpoint, input validation that returns a proper `ValidationProblemDetails` body instead of a raw 500, and global exception middleware so an unhandled exception still comes back as a structured `ProblemDetails` response, not a stack trace.

The refactor exercise: reading bad code on purpose

Given a deliberately bad `OrderController` – one giant POST endpoint mixing validation, business logic, data access, and response shaping; empty `catch {}` blocks; synchronous EF Core calls inside an `async` method; an `object` return type; an off-by-one bug; multiple `SaveChanges()` calls; zero tests – and wrote up every smell before touching a fix.

INTERVIEW ANGLE

The point of this exercise isn't "write it in a service layer instead" – it's naming *why* each smell is a real cost. An empty `catch` block doesn't fail loudly, it fails silently, which is strictly worse than crashing. A sync call inside an `async` method blocks a thread pool thread for the duration of I/O, which is the opposite of what `async` is for. One class doing validation, business rules, and persistence means one change (a new validation rule) risks breaking an unrelated concern (how the response is shaped) – the textbook argument for single responsibility.

DAY 02 **Dependency injection, a rich domain model, and auth that survives a stolen token**

An `IClock` abstraction, and why DI lifetimes matter

Replaced every direct call to `DateTime.UtcNow` with an injected `IClock`, registered as a Singleton, and wired it through `Collection` → `CollectionItem` → the endpoint. Proved it works with a `FakeClock` in a test that pins timestamps to an exact, controllable value.

INTERVIEW ANGLE

`DateTime.UtcNow` is a hidden dependency – code that calls it directly can't be tested deterministically, because the answer changes every time you run the test. Wrapping it in `IClock` makes time an injectable value like any other dependency. Singleton is the right lifetime here specifically because a clock has no per-request state – Scoped or Transient would just add allocation overhead for no benefit.

Proving `CancellationToken` actually cancels something

Wrote an integration test using `WebApplicationFactory` that cancels a token mid-request against the real endpoint→repository→EF Core chain, and confirmed the operation does not silently complete anyway.

INTERVIEW ANGLE

A `CancellationToken` only cancels anything if every layer actually threads it through and every awaited call actually honors it – it's not automatic. It's easy to write code that accepts a token parameter and never passes it anywhere, which compiles fine and does nothing. Testing this with a real in-memory HTTP client is the difference between "the signature looks right" and "the behavior is right."

Testing the domain layer in complete isolation

Pure xUnit + FluentAssertions tests for the `Collection` aggregate's own rules – empty name throws, name over 80 characters throws, a 51st item throws, a duplicate quote ID throws, removing a non-existent item throws – with no database and no HTTP involved at all.

VERIFIED

6 / 6

domain invariant tests passing, zero infrastructure

Anemic model → rich domain model

Refactored `Quote` from plain public properties into a rich model: private setters, a static factory (`Quote.Create(author, text)`) that returns a `Result<Quote>` instead of throwing, and a soft-delete flag instead of physically removing rows. Text can never change after creation – only the row can be marked deleted.

INTERVIEW ANGLE

An anemic model is a data bag – anything can set any field to anything, so the "rules" live scattered across every caller instead of in one place. Moving construction behind `Quote.Create()` means there is exactly one place an invalid `Quote` could come from, and that place already refuses to produce one. Returning a `Result` instead of throwing keeps "this input was invalid" as an ordinary, expected return value instead of an exception used for control flow.

JWT auth: login, hashed passwords, protected writes

Added a real login endpoint – email/password in, a signed JWT access token and a refresh token out. Passwords are hashed with BCrypt.Net-Next, never rolled by hand. `POST` and `DELETE` on `/api/quotes` now require a valid token; `GET` stays open.

INTERVIEW ANGLE

Access tokens are short-lived (15 minutes here) and signed, so the server never has to look them up to trust them – that's the whole point of a JWT. Refresh tokens are long-lived, opaque random strings with no meaning of their own; their entire job is to be exchanged for a new access token without asking the user to log in again every 15 minutes.

Refresh token rotation with reuse detection

Every refresh token is single-use: calling `/api/auth/refresh` mints a brand-new access/refresh pair and immediately revokes the token that was just presented, linking the old one to its replacement. Tokens are stored in the database as a SHA-256 hash, never in plain text. If a token that's already been revoked is presented again, that is treated as a signal the token was stolen – the entire refresh-token family for that user is revoked at once, forcing every session, legitimate or not, to log in again.

INTERVIEW ANGLE

This is the strongest single interview story from Day 2. Storing a hash instead of the raw token means a stolen database dump doesn't hand out usable refresh tokens, the same reasoning as hashing passwords. Rotation means a stolen refresh token is only useful once – the moment the real user's app refreshes normally, the stolen copy becomes a "revoked" token. Reuse detection is what turns that into an actual defense rather than a footnote: if an attacker refreshes with their stolen copy *before* the real user does, the real user's next refresh attempt presents a token that's already been revoked and replaced – which is exactly the reuse signal that revokes the whole chain. Neither side gets to quietly keep using a compromised token.

DAY 03 A second identity provider, real authorization, and CI

Microsoft Entra ID alongside the self-issued JWT

Added Entra ID as a second identity provider that works on the exact same endpoints as the Day 2 JWT scheme, with zero changes to endpoint code. Three authentication schemes are registered: `Internal` (validates the app's own HS256 tokens), `Entra` (validates tenant-issued tokens, pulling signing keys live from the tenant's JWKS endpoint), and a `PolicyScheme` set as the default that decides which of the other two should handle a given request.

INTERVIEW ANGLE

A policy scheme is a router, not a validator. It reads the incoming token's `iss` (issuer) claim *without* validating the signature, and forwards to whichever scheme can actually verify it – tokens from `login.microsoftonline.com` or `sts.windows.net` go to `Entra`; everything else, including malformed tokens, falls through to `Internal`, which rejects them with an ordinary 401. It also had to tolerate both Entra token generations – v1 tokens use the `sts.windows.net` issuer and an `api://{guid}` audience, v2 tokens use a `/v2.0` issuer and a bare GUID audience.

Claim-based policies and resource-based ownership

Added three claim policies – `can-read-quotes`, `can-edit-quotes`, `can-delete-quotes` – applied declaratively to endpoints. Because the two schemes carry scopes differently (self-issued tokens use a `scope` claim directly; Entra uses `scp` for delegated permissions and `roles` for app-only ones), an `IClaimsTransformation` normalizes both into one `scope` shape once per request, so the policies never need to know which scheme authenticated the caller. "Can this caller delete *this specific* quote" can't be answered from the token alone, since it depends on a database row – that check runs imperatively, after the quote is loaded, via `IAuthorizationService`.

INTERVIEW ANGLE

401 means "I don't know who you are"; 403 means "I know exactly who you are, and the answer is no." That distinction decided the endpoint's order of operations: the delete endpoint loads the quote *first*, so a missing quote correctly returns 404 and an unowned one returns 403 – reversing the order would leak whether a quote exists to a caller who isn't even allowed to delete it. Policies over hardcoded roles matters too: `RequireClaim` reads as "does this token carry this permission," which is portable across identity providers; `RequireRole("admin")` hardcodes a role name that may not even exist in a second provider.

Testcontainers and a real CI pipeline

Swapped the integration tests' SQLite fixture for a disposable real SQL Server instance spun up per test run via Testcontainers, so tests run against the actual target database engine instead of a close-enough substitute. Added a GitHub Actions workflow that restores, builds, and runs the test project on every push and pull request.

INTERVIEW ANGLE

SQLite and SQL Server don't always agree – different default collations, different handling of certain data types, different locking behavior entirely. A test suite that passes on SQLite but exercises SQL-Server-specific authorization logic isn't actually proving the production database will behave the same way. Testcontainers closes that gap by testing against a real, disposable instance of the real engine, at the cost of a slower test run.

DAY 04 Making the API observable, and provably correct at startup

OpenTelemetry tracing + Serilog, unified on one ID

Wired OpenTelemetry tracing with ASP.NET Core and EF Core instrumentation, and Serilog for structured logs enriched with a correlation ID via `LogContext`. The two were initially pulling their "trace ID" from two *different* sources – a custom `HttpContext.TraceIdentifier` versus OpenTelemetry's own `Activity.Current` – so the console log line, the `X-Trace-Id` response header, and the actual trace span didn't always agree. Fixed by making every one of them read from `Activity.Current.TraceId`.

INTERVIEW ANGLE

The whole point of a correlation ID is that a user or a support ticket can hand you one value and you can find everything about that request – logs, spans, and the response header – with it. Two systems generating "the same" ID independently is a trap: they only coincidentally match until the exact moment they don't, which is precisely when you need them most. This was caught with a test that asserts the *actual rendered log field* equals the response header, not just that both features were switched on.

Typed configuration that fails at startup, not at 2am

Replaced raw `IConfiguration["Jwt:Key"]` string lookups with typed `JwtOptions` / `EntraOptions` records carrying `[Required]` attributes, registered with `ValidateOnStart`.

INTERVIEW ANGLE

Without validation, a missing `Jwt:Key` in configuration doesn't fail until the first request that happens to need it – which might be minutes or hours after a bad deploy goes live. `ValidateOnStart` makes the exact same mistake throw immediately when the host starts, so a broken config never serves a single request. Verified by booting a real generic `Host` through the production startup path and asserting it throws for a missing key, an empty-string key, and a missing tenant ID – and does not throw when everything is valid.

Azure Monitor, KQL, and an actual alert condition

Added a config-gated Azure Monitor trace exporter (off by default so forty-plus integration tests that each boot a real host don't try to call Azure) and wrote real Kusto (KQL) queries: the ten slowest requests in the last hour, a full `operation_Id` timeline joining requests/dependencies/traces for one request, and an alert query for the average duration of `POST /api/quotes` over a five-minute window.

INTERVIEW ANGLE

`operation_Id` in Application Insights is the same trace ID that comes back on the response header – that's the direct answer to "how would you find everything Azure Monitor knows about one slow request a customer reported." And the alert-condition query is the direct answer to "how would you get paged when quote creation gets slow": average duration on a named operation, over a rolling window, not a single-request threshold that would false-positive on any one slow outlier.

CI coverage gate: 70% → 80%

Extended CI to collect Cobertura coverage from both unit and integration test projects, merge the reports, and fail the build if line coverage drops below a threshold. Started at 70%; after extracting a tangled `AuthenticationExtensions` class and deleting genuinely dead code, coverage reached 99.8% and the gate was raised to 80%.

VERIFIED

99.8%

line coverage reached

70%→80%

CI gate raised after cleanup

DAY 05 Diagnosing a real N+1 bug with tracing, then shipping the container

Finding N+1 the way you would in production: by tracing, not by reading code

Deliberately regressed `CollectionRepository.GetByIdAsync` to load a collection, then loop over every item issuing a *separate* database query per item – and found it the way a real incident would surface it: by looking at the trace, not by reading the code and guessing. Fixed by replacing the loop with a single `.Include(c => c.Items)` query.

VERIFIED – BEFORE VS. AFTER, SAME 20-ITEM COLLECTION

CONSOLE TRACER OUTPUT, POST `/COLLECTIONS/{ID}/ITEMS`

	DB spans	Total spans	Duration
Before (N+1)	23	24	6.62 ms
After (.Include)	3	4	2.63 ms

INTERVIEW ANGLE

N+1 is "one query to get a list, then one more query per item in that list" – 20 items means 20 extra round trips to the database for work a single join could do in one. It's dangerous specifically because it scales with data, not with code changes: it works fine in a demo with three rows and gets silently worse as real data grows, which is exactly why it needs to be caught by tracing (which shows the real shape of what happened) rather than by code review alone.

Containerizing the API – and two production-shaped bugs in one afternoon

Used .NET's built-in `dotnet publish /t:PublishContainer` support (no hand-written Dockerfile) targeting an Alpine base image, plus a real `/health` endpoint that deliberately allows anonymous access, since a container orchestrator's health probe carries no bearer token.

TWO BUGS CAUGHT, IN SEQUENCE, BOTH FROM REAL CRASH LOGS

1. glibc vs. musl. Publishing for `linux-x64` (glibc) onto an Alpine base (musl) broke SQLite's native library before the app could even bind a port – `DllNotFoundException`, a missing `fcntl64` symbol. Fixed by publishing for `linux-musl-x64` instead; the correct native binary was already in the NuGet package, it simply wasn't being selected.

2. A non-root filesystem. After fixing #1, the container still crashed on `db.Database.Migrate()` – "unable to open database file." .NET's container images run as a non-root user, and that user doesn't own `/app`. Fixed with zero code changes: pointed the SQLite connection string at `/tmp` (world-writable) via an environment variable at run time.

VERIFIED

59.2 MB

final image size

200 / 200 / 401

/health, GET quotes, unauthenticated POST – all correct

Deploying to Azure Container Apps – a third, mirror-image RID bug

Provisioned an Azure Container Apps environment and deployed with `azd`. Because `azd`'s default publish path targets glibc, the same Alpine base that worked locally broke again in the cloud – this time the base image had to move to a glibc-compatible Debian image rather than away from it.

INTERVIEW ANGLE

Same root cause, opposite direction, twice in one day – that's the actual lesson: "which C library does this container's base image use, and does that match what was published for" is a question worth asking *before* a deploy, not after reading a crash log, once you've been bitten by it once.

Resilience with Polly: timeout, retry, circuit breaker – in that order

Wrapped an outbound call to a third-party quotes API in a Polly pipeline, composed outermost-to-innermost as: total timeout (10s) → retry (3 attempts, exponential backoff with jitter) → circuit breaker (opens at a 50% failure ratio over a rolling window) → the actual HTTP call.

INTERVIEW ANGLE

Order isn't arbitrary. Timeout has to wrap retry, or a slow dependency could let three retries each hang for a long time with no overall cap on wall-clock time. Retry has to wrap the circuit breaker, so that once the breaker trips open, the retry loop stops dead immediately instead of continuing to hammer a dependency that's already known to be down. Verified with tests: two failures then a success retries exactly twice and succeeds; four straight failures exhausts all three retries and fails fast rather than hanging; and with the throughput threshold lowered for the test, the breaker demonstrably opens after repeated failures and the next call is rejected without even reaching the dependency.

PART TWO

Data & Performance Engineering

A deliberate drop below the ORM, into SQL Server itself: CTEs and window functions, what an index physically is, transaction isolation and a real deadlock, what EF Core does behind the scenes, and profiling a slow endpoint until the fix is provable, not just plausible.

Day 7 – CTEs, windows, set ops

Day 8 – Indexing

Day 9 – Isolation & deadlocks

DAY 07 SQL beyond SELECT: CTEs, window functions, set operations

CTEs instead of a correlated subquery

One query returning each author with their quote count and their most recent quote, built from two named CTEs – one grouping and counting, one ranking each author's quotes by recency with `ROW_NUMBER() OVER (PARTITION BY Author ORDER BY Id DESC)` – joined together on rank 1.

INTERVIEW ANGLE

A correlated subquery in the SELECT list re-runs once per output row – the same N+1 *shape* as Day 5's EF Core bug, just pushed down inside the database where a trace won't show it as separate spans. The CTE version scans the table twice, total, no matter how many authors exist. Naming this out loud – "this is N+1 logic, it's just hidden inside one SQL statement" – is a strong answer to "how do you spot a correlated subquery problem."

Window functions: `ROW_NUMBER`, `running SUM`, `LAG`

Per author, computed a running quote count and the number of days since that author's previous quote – `ROW_NUMBER()` for position, `SUM(1) OVER (...)` for the running total, and `LAG(CreatedAt)` to look back one row – all partitioned by author so each author's numbering restarts independently in a single pass over the table.

INTERVIEW ANGLE

`PARTITION BY` is what separates a window function from a plain aggregate – it resets the window per group without collapsing the rows the way `GROUP BY` would. Worth naming the subtle trap found here: the first row per author has no previous quote, so `LAG` returns `NULL` – not a bug, but a real downstream risk, because `AVG()` over that column silently *skips* null rows rather than erroring, quietly shrinking the denominator for authors with few quotes.

Set operations mapped straight onto business questions

Three questions, each answered with the SQL operator that says it most directly: authors with quotes but no tags via `EXCEPT`, authors present in two different tag categories via `INTERSECT`, and a combined distinct tag list via `UNION`.

INTERVIEW ANGLE

All three could be written with `NOT EXISTS`, a join, and `DISTINCT` – set operators aren't faster, they're clearer. "Authors with quotes but no tags" is literally a subtraction, and writing it as `EXCEPT` lets the next reader see the intent immediately instead of reconstructing it from a `NOT EXISTS` clause. The one real cost worth mentioning: `UNION` de-duplicates, which requires a sort – `UNION ALL` is cheaper when duplicates genuinely don't matter.

DAY 08 Indexing: what actually happens on disk

Clustered vs. non-clustered, measured with real IO stats

Seeded 100,000 rows with deliberately varied selectivity and ran four fixed queries unchanged across four index stages, capturing logical reads with `SET STATISTICS IO ON` at each stage.

VERIFIED – LOGICAL READS PER STAGE

Query	Heap	Clustered	+ Non-clustered	+ Covering
Point lookup (unique key)	529	3	3	3
Ranged filter (medium selectivity)	529	565	33	33
Aggregate / GROUP BY	529	565	347	2
Low-selectivity filter (3 distinct values)	529	565	565	565

INTERVIEW ANGLE

The clustered index *is* the table – its leaf level physically stores the sorted data rows, and a table can only have one. A non-clustered index is a separate, slimmer structure of just the key plus a pointer back to the base table; fetching anything else means a "key lookup" back into the clustered index, which is expensive per row. The table above is worth walking through directly: adding *just* a clustered index made three of the four queries *worse* (529→565 reads), because an ordered scan still reads every row – it isn't free. Only a matching non-clustered index actually helped. And the low-selectivity column (three distinct values across 100,000 rows) never improved no matter what index was added – proof that indexing a column with too few distinct values is pointless; the optimizer correctly refuses to use an index that wouldn't narrow the row set enough to beat a scan.

Covering indexes: removing the key lookup entirely

Converted a plain non-clustered index into a covering index by adding `INCLUDE (DurationMs)`, and captured the execution plan before and after.

VERIFIED

347 → 2

logical reads (173x fewer) after adding INCLUDE

Lookup gone

plan collapses from Seek+Lookup to a single Index Seek

INTERVIEW ANGLE

INCLUDE stores extra columns at an index's leaf level without making them part of the sort key – they don't affect the seek predicate, they're just sitting right there when the engine reaches the row it already found. A covering index is one that contains every column a query needs, so the key-lookup step disappears from the plan entirely. This is the single cleanest before/after number across the whole two weeks – worth having ready as the answer to "tell me about a time indexing actually mattered."

DAY 09 Transactions: isolation levels, the three anomalies, and a real deadlock

Reproducing all three classic read anomalies

Each anomaly reproduced with two literal, interleaved SQL sessions talking past each other against the same table.

Anomaly	What happens	Prevented starting at
Dirty read	Session B reads Session A's uncommitted change; A then rolls back – B read data that never existed	READ COMMITTED
Non-repeatable read	B reads a row twice in one transaction; A commits a change in between – B gets two different values for the same row	REPEATABLE READ
Phantom read	B re-runs the same range query twice in one transaction; A inserts a new row into that range – B's row count changes mid-transaction	SERIALIZABLE

INTERVIEW ANGLE

This is a strict ladder, worth stating in order: **READ UNCOMMITTED** prevents nothing; **READ COMMITTED** (the default) prevents dirty reads; **REPEATABLE READ** also prevents non-repeatable reads by locking a row on first read; **SERIALIZABLE** also prevents phantoms by taking a range lock, not just a row lock. Each level up trades more locking (and therefore more blocking, less concurrency) for a stronger correctness guarantee – there's no free level that prevents everything for free.

A real deadlock, reproduced and fixed on purpose

Two sessions each updated the same two rows in *opposite order* – Session A: row 1 then row 2; Session B: row 2 then row 1 – both left open. SQL Server's deadlock monitor detected the cycle and killed one session as the "victim," with the actual deadlock graph captured as evidence.

FIX

The exact same two transactions, but with both sessions made to acquire locks on the shared rows in the *same order* (row 1, then row 2) – which removes the possibility of a circular wait entirely, rather than trying to detect or retry around it.

INTERVIEW ANGLE

A deadlock happens when two transactions each hold a lock the other one needs, and neither can proceed. SQL Server doesn't hang forever – its deadlock monitor detects the cycle and kills one transaction (the one estimated to be cheapest to roll back), which fails with error 1205. The textbook fix is exactly what was applied: make every piece of code that touches the same set of resources acquire locks in a consistent order, so a cycle can never form in the first place.

DAY 10 What EF Core is actually doing on your behalf

The change tracker, and what `AsNoTracking()` actually buys

Measured tracked reads against `AsNoTracking()` reads on a 10,000-row table: identity behavior (repeated tracked reads return the exact same object instance; `AsNoTracking()` returns a fresh one each time), and raw timing/allocation.

VERIFIED – 10,000-ROW READ, FRESH DBCONTEXT, AFTER A DISCARDED WARM-UP

	Elapsed	Allocated
Tracked (default)	46 ms	9.83 MB
<code>AsNoTracking()</code>	12 ms	3.87 MB

INTERVIEW ANGLE

EF Core's change tracker keeps an "identity map" – one tracked object per row per `DbContext` – plus a snapshot of each entity's original values, purely so `SaveChanges()` can later compute what changed. Read-only code pays for that bookkeeping and gets nothing from it. `AsNoTracking()` skips both, at roughly 2.5x less memory here, for exactly the case where it's free to skip: data that's only ever going to be displayed, never written back.

Projections: only pull the columns you actually use

Querying a whole `Quote` entity generates a SELECT of every mapped column, including ones nothing reads – EF has to materialize a complete, trackable entity. Projecting through `.Select(q => new QuotedTo { { ... } })` shrinks the generated SQL to only the requested columns.

The client-eval trap – and why modern EF Core refuses it outright

Calling an ordinary local C# method inside a LINQ-to-EF predicate threw `InvalidOperationException` immediately – before any SQL was even sent – because the expression couldn't be translated to SQL.

THE GOTCHA, AND HOW IT WAS ACTUALLY FIXED

"Client evaluation" means part of a LINQ query can't become SQL and instead runs in .NET memory. In EF6, this used to happen *silently* – the whole table would get pulled into memory to run arbitrary C# against it, with no warning, which is a severe, invisible performance cliff on a large table. Modern EF Core refuses instead of doing that quietly. The real fix was rewriting the predicate into something SQL can express directly (`q.Author.Length < 8` , which compiles to `WHERE length(Author) < 8`) so filtering still happens in the database – not just working around the exception with `.AsEnumerable()` , which technically "fixes" the crash but quietly drops the WHERE clause and pulls every row before filtering in memory, reintroducing the exact problem EF was refusing to hide.

INTERVIEW ANGLE

This is a genuinely good "what's a surprising EF Core behavior you've hit" answer: the framework getting *stricter* over time (throwing instead of silently degrading) is a real, deliberate design change worth knowing about, not just a rule to memorize.

DAY 11 Profiling a slow endpoint until the fix is provable

An endpoint that's bad two ways at once

Built `/authors/slow` : one query for distinct authors, then a separate query *per author* inside a loop, sorted on a column with no index – so N+1 and a missing index compound each other, since every one of those N queries also does a full table scan.

VERIFIED – LOAD TEST AGAINST `/AUTHORS/SLOW`

295.72 ms **785.62 ms** **~31 req/s**
p50 latency p99 latency throughput

The fix, built alongside the slow version for direct comparison

Added `/authors/fast` – one `GROUP BY` query instead of a loop – plus a real index on the filtered column, without removing the slow version, specifically so the two could be compared side by side.

VERIFIED – EXECUTION PLAN COST, SAME QUERY, THREE INDEX STATES

	Subtree cost
Table scan (no index)	8.27E-2
Non-covering index (key lookup per row)	8.43E-2
Covering index (no lookup)	1.49E-2

INTERVIEW ANGLE

The middle row is the one worth explaining unprompted: a plain non-covering index made the query *barely* better, almost indistinguishable from the scan, because SQL Server still does a key lookup back to the clustered index for every matching row. The covering index dropped the cost about 5.6x by removing that lookup step from the plan entirely – the same pattern as Day 8's covering index result, at smaller scale, which is exactly the kind of repeated, provable pattern worth naming as "this isn't a one-off, I saw the same shape twice."

DAY 12 Splitting reads from writes, and Dapper vs. EF Core

CQRS-lite: commands and queries as two separate, honest paths

Split the `Collections` feature into a command path (`CreateCollectionCommandHandler` – validates, enforces invariants, touches the domain aggregate) and a query path (`CollectionSummaryQueryHandler` – a single `AsNoTracking()` EF projection joining `Collections` → `CollectionItems` → `Quotes` straight into a DTO, no domain entity materialized at all).

INTERVIEW ANGLE

"CQRS-lite" means taking the core idea of Command Query Responsibility Segregation – reads and writes have different needs, so don't force them through the same code path – without adopting a full mediator or message-bus framework. A query can be an aggressively denormalized, shaped-for-the-screen projection with zero risk to write-side rules, precisely because a query result is never saved back through the write path. A test here explicitly asserts the query path emits exactly one SQL statement, which is the real point: a query handler should not accidentally reintroduce Day 5's N+1 shape.

Dapper alongside EF for the read path

Added a second implementation of the same query, hand-written SQL mapped by Dapper instead of EF's LINQ translation, and benchmarked both against each other while asserting they return identical results.

VERIFIED – 1,000 SEQUENTIAL WARM CALLS, SAME SEEDED DATA

	Total	Per call
EF Core	284.16 ms	0.284 ms
Dapper	60.16 ms	0.060 ms

A REAL MAPPING GOTCHA, NOT A HYPOTHETICAL ONE

SQLite returns integer columns as `Int64`, and Dapper's constructor-based mapping needs an exact type match – so the read DTO had to become a plain mutable class with settable properties instead of a record, because Dapper widens into settable properties correctly but doesn't reliably match a record's constructor signature the same way.

INTERVIEW ANGLE

About 4.7x faster per call here, and the honest reason is worth stating precisely: Dapper skips LINQ-expression-tree evaluation, query-plan-cache lookups, and any change-tracking-adjacent materialization – even under `AsNoTracking()`, EF still walks an expression tree first. The tradeoff is real, not just a footnote: hand-written SQL has no compile-time link to the schema, so a column rename won't be caught until the query runs, and it took two round trips here versus EF's one multi-join statement. This is a "know when to reach for it" answer, not a "Dapper is just better" one – the docs for this benchmark explicitly caveat it's one run on one machine.

PART THREE

The Angular Frontend

Building the client for the same API from a blank Angular 21 project: signals instead of Zone.js, two different ways to build the same form, accessibility checked live instead of assumed, and – on the last day – directing an AI coding agent and reviewing its output the way you'd review a junior engineer's pull request.

Day 13 – Signals, zoneless

Day 14 – Forms & live a11y

DAY 13 Angular from scratch: signals, not Zone.js

A zoneless app, built on signals from the first line

Built the Angular 21 frontend for the same QuotesApi with `provideZonelessChangeDetection()` – no zone.js at all – using `signal()`, `computed()`, and `effect()` for the quotes list and detail view, modeled explicitly as a loading / loaded / error state machine.

INTERVIEW ANGLE

Classic Angular change detection relies on Zone.js patching almost every async browser API (`setTimeout`, `fetch`, event listeners) and then re-checking the *entire* component tree whenever any of them fire. Signals flip that: each signal knows exactly which parts of the template read it, so only those parts re-render, and Zone.js's global monkey-patching isn't needed at all. That's both a performance change and an architectural one – it's why zoneless apps have to opt in deliberately rather than just deleting a dependency.

A real race condition, caught by clicking fast

The quote-detail component re-fetches whenever the selected quote ID changes. If a user selects a second quote before the first request finishes, the two responses can arrive out of order – a stale-response guard discards any response that resolves after a newer selection has already been made, instead of letting whichever response arrives last silently win.

INTERVIEW ANGLE

This is worth having ready for "tell me about a race condition you've actually hit" – it's not a network failure, it's ordinary fast clicking, and it's invisible in a demo where nobody clicks quickly. The fix pattern is simple and general: capture the identifier the request was made for, and when it resolves, check that identifier is still the current one before applying the result.

DAY 14 Two ways to build the same form, and why accessibility has to be checked live

Reactive Forms and Signal Forms, side by side, on purpose

Built the "Create quote" form twice: once with Angular's established Reactive Forms (`FormGroup` / `FormControl` / `Validators`), and once with `@angular/forms/signals` – an experimental preview API doing the same job on top of signals instead of an Observable-based control tree.

A REAL BUG, ONLY FOUND BY TYPING INTO THE FORM

Submitting the Signal Forms version with a whitespace-only author showed a doubled error – "Author is required. Author is required." – because two separate validators both fired for the exact same input: the built-in required check and a custom whitespace check that hadn't excluded the case the required check already owned. Fixed by guarding the whitespace validator to skip the exact-empty-string case. Not caught by reading the validator code – only by actually typing a space into the field in a real browser.

INTERVIEW ANGLE

Reactive Forms and Signal Forms solve the same problem with different underlying primitives – one is Observable-based, the newer one is signal-based – but validator composition bugs like this one aren't specific to either; they're a general lesson about layering multiple validators over the same input without checking whether their empty cases overlap.

Accessibility, verified live, not assumed

Ran an automated WCAG audit against the real rendered DOM – Playwright driving the system-installed Chrome, with axe-core doing the actual scan – rather than trusting that using semantic HTML and `aria-*` attributes was sufficient on its own.

INTERVIEW ANGLE

Semantic markup is necessary but not sufficient – focus order, live announcement of dynamic errors, and correct `aria-describedby` wiring are exactly the kind of thing that can look right in the template source and still fail once something actually queries the live, rendered accessibility tree. "I ran an automated scanner against the real DOM" is a materially stronger answer than "I used semantic HTML."

A visual redesign, done twice – because the first pass wasn't actually a redesign

The first redesign attempt only changed CSS color tokens and was correctly called out as not being a real change. The second pass changed typography, layout, and motion – a serif display face, a real hero section, staggered entrance animation, a responsive nav – which is what a redesign actually has to touch to register as one.

INTERVIEW ANGLE

Worth naming directly: a color-only pass is a theme change, not a redesign. Structure, type, and motion are what a reviewer is actually asking about when they say "make it look different."

DAY 15 Directing an AI agent – and reviewing its diff like a PR

The task, and the actual point of the exercise

The stated goal: attach an auth header to every request, retry failed GET requests automatically, and turn raw server error JSON into a friendly on-screen message. The real exercise underneath it was directing an AI coding agent (Claude Code) and verifying its output the way a senior engineer reviews a junior's pull request – not typing the interceptors by hand.

Step one: a characterization test, green before any UI code existed

Before writing a single interceptor, wrote a test that pins down exactly what the real backend already returns today – the shape of `GET /api/quotes`, and the real

`ValidationProblemDetails` body on a 400 – and ran it green against the existing, unmodified service code.

INTERVIEW ANGLE

A characterization test doesn't test that behavior is "correct" – it pins down what the system *actually does right now*, so any later change has a fixed, real target to build against instead of an assumed one. Directing an agent to write this first, and confirming it passes before any new code is written, is what "green before any UI" actually buys: proof the new work is being built against reality, not against a guess at the contract.

Retry with backoff – and knowing exactly when not to retry

A GET request that fails with no response at all, or a 5xx, is retried up to twice with growing delays (300ms, then 600ms). A GET that fails with a 4xx is never retried at all.

INTERVIEW ANGLE

The "never retry a 4xx" rule is the part worth explaining, because it's not obvious until you say it out loud: a 400 or a 401 will fail *identically* on every attempt – the request itself is wrong, not the network – so retrying it just adds delay before showing the same error the first attempt would have shown immediately.

Typed error mapping: one consistent shape for every failure

Added a function that turns the backend's raw error JSON – which takes two different shapes depending on the endpoint – into one consistent object with a plain-English message, so every screen just reads `error.friendlyMessage` instead of each one re-parsing the server's response format independently.

CAUGHT WHILE REVIEWING THE DIFF, BEFORE ACCEPTING IT AS DONE

The characterization test only proved the existing API-calling code still worked – it never touched the new error-mapping logic at all, which had zero test coverage of its own. A dedicated test file for just that function was written and required before the interceptor work was accepted, covering every real error shape the backend actually sends: field-level validation errors, a login failure with no body at all, and a permission-denied response.

INTERVIEW ANGLE

This is the actual argument for "why does directing an AI agent still require real review": a test suite passing green is not the same claim as "the new code is tested" – it's easy for a green suite to be testing everything *except* the new logic, and that gap only shows up if someone reads the diff and asks what each test actually covers.

A UI bug caught only by watching the browser, not by any test

A "Quote added" success message was being set correctly in code, but could never actually appear on screen – the parent component switched views in the exact same instant it told the child to display that message, so the message never got a chance to render before the screen changed underneath it.

FIX AND RE-VERIFICATION

Added a short delay before switching views, specifically so the confirmation is visible first, then re-verified live in a real browser that the message now actually renders before the app navigates away.

INTERVIEW ANGLE

None of the twelve automated tests written that day touched this component's view-switching logic at all – this bug could only ever be found by actually watching the app run, which is the concrete argument for why "the tests pass" and "I verified it works" are not the same sentence.

Proving the retry logic really works – not just trusting the source

Rather than reading the interceptor's code and assuming it was correct, forced the server to return a temporary error and counted the real network requests a browser actually made.

VERIFIED LIVE, AGAINST A REAL BROWSER

3 requests

on a forced 503 (1 original + 2 retries)

1 request

on a forced 404 (no retry, as designed)

How to talk about all fifteen days

One thesis, and four go-to stories – the ones that hold up best when someone actually asks a follow-up question.

THE THREAD THAT TIES EVERY DAY TOGETHER

Every real fix in these fifteen days was found and proven with a measurement – a trace, an execution plan, a load test, or a live browser check – never just asserted from reading the code. That's the one sentence worth saying if asked to summarize the whole thing.

01 The N+1 bug, found by tracing (Day 5)

23 database spans collapsed to 3, 6.62ms to 2.63ms, found by reading a trace rather than the code – the strongest "how do you debug performance in production" answer of the fifteen days.

02 Refresh token rotation with reuse detection (Day 2)

Hashed, single-use refresh tokens where presenting an already- revoked token revokes the entire session family – the strongest security-design story, with a clear "why" for every design choice.

03 The covering index, twice (Days 8 and 11)

A 173x and a 5.6x reduction in query cost from the same fix – removing the key-lookup step by including the needed column – seen twice at different scales, which is what makes it a pattern rather than a coincidence.

04 Reviewing an AI agent's work like a PR (Day 15)

A green test suite that quietly never tested the new logic, and a UI bug no test could have caught – both only surfaced because the diff was read and the app was actually watched, not because the agent was trusted.

Covers Day 1 through Day 15 of the thinkschool engineering exercises – ASP.NET Core / EF Core / SQL Server backend (Days 1-12) and an Angular 21 frontend against the same API (Days 13-15). Figures and file/endpoint names throughout are taken directly from the project's commit history, test output, and documentation rather than approximated.